

THE JUNAGADH COMMERCIAL CO-OP. BANK LTD.

Phase 1: Architecture & Firestore Read Surge Audit Report

Central Core System Migration to Supabase (PostgreSQL)

Document ID: JCCB-MIG-PHASE1
Classification: Confidential - Internal Banking
Audit Date: September 02, 2026

ACTIVE TERMINALS	OBSERVED SURGE	CURRENT DB ENGINE	TARGET ARCHITECTURE
120+ Employees (17 Branches)	123,000+ Reads (in 48 Hours)	Firestore + Firebase Auth	Supabase (PostgreSQL + RLS)

Executive Audit Finding: The 123,000+ Firestore read spike is the direct mathematical consequence of **unmanaged client-side polling loops running at 5-second intervals across 120+ browser sessions**, combined with unbounded real-time snapshot listeners downloading entire database collections without server-side filtering or pagination.

1. Root Cause Breakdown & Ranked Offenders

Forensic inspection of the production client codebase (`app_gold.js` , `management.js` , `firebase-config.js`) identified six architectural root causes:

RANK	OFFENDER & LOCATION	OPERATIONAL BEHAVIOR	CALCULATED IMPACT	SEVERITY
#1	5-Second Global Sync Loop app_gold.js (L696) syncCloudData()	Every 5,000ms, each open browser tab pulls <code>loans</code> , <code>deleted_loans</code> , <code>rates</code> , and 5 separate master setting docs.	~691,200 poll cycles/day across 120 client PCs. Reads entire <code>loans</code> collection on every cycle.	CRITICAL (Top Driver)
#2	Session Heartbeat Read-Check app_gold.js (L1307) firebase-config.js (L1305)	Every 30 seconds, <code>updateDeviceHeartbeat</code> runs a <code>doc.get()</code> read on <code>active_sessions/{sessionId}</code> before updating state.	~345,600 document reads/day across 120 client terminals purely for session heartbeat pings.	CRITICAL
#3	Unbounded Live Listeners app_gold.js (L746) listenLoans()	Every branch tab subscribes to the entire multi-branch <code>loans</code> collection via <code>onSnapshot()</code> without query limits or branch scoping.	Exponential read multiplier: 1 write in any branch pushes reads to all 120+ connected browsers.	HIGH
#4	Management Portal 5s Poller management.js (L202) loadAllData()	Executes <code>Promise.all()</code> across 6 collections/docs (including full <code>loans</code> and <code>audit_logs(300)</code>) every 5 seconds.	~17,280 queries/day per open central management console window.	HIGH
#5	Zero Server-Side Scoping firebase-config.js (L529)	Branch operators download all 17 branches' loans over the network, only to filter them in JavaScript client memory.	17× unnecessary data over-fetching per terminal. Zero tenancy isolation.	MEDIUM
#6	Discarded Unsubscribes app_gold.js & management.js	The return values of <code>onSnapshot()</code> are never stored or called on re-authentication, causing listeners to stack up in memory.	Continuous listener accumulation across shift changes and terminal re-logins.	MEDIUM

2. Complete Inventory of All Firestore Reads

A. Real-Time Snapshot Subscriptions (`onSnapshot`)

TARGET COLLECTION / DOC	SOURCE FILE & LINE	QUERY SCOPE & LIMITS	LIFECYCLE UNSUBSCRIBE HANDLED?
<code>loans</code>	<code>app_gold.js:746</code>	Entire collection (Unfiltered across all branches)	No Cleanup (Memory leak)
<code>deleted_loans</code>	<code>app_gold.js:719</code>	Entire collection (Unbounded tombstone sync)	No Cleanup
<code>customers</code>	<code>app_gold.js:908</code>	Entire collection (Unbounded directory)	No Cleanup
<code>rates/today</code>	<code>app_gold.js:705</code>	Single document	No Cleanup
<code>settings/branchSeeds</code>	<code>app_gold.js:793</code>	Single document	No Cleanup
<code>settings/rulesMaster</code>	<code>app_gold.js:811</code>	Single document	No Cleanup
<code>settings/branchesList</code>	<code>app_gold.js:825</code>	Single document	No Cleanup
<code>settings/valuerslist</code>	<code>app_gold.js:861</code>	Single document	No Cleanup
<code>settings/productsList</code>	<code>app_gold.js:896</code>	Single document	No Cleanup
<code>active_sessions/{sid}</code>	<code>app_gold.js:1340</code>	Single document (Re-attached on every login)	No Cleanup (Duplicates accumulate)
<code>active_sessions</code>	<code>management.js:151</code>	Entire collection (Realtime live terminals)	No Cleanup
<code>audit_logs</code>	<code>management.js:162</code>	Order by <code>timestampMs desc</code> , Limit 200	No Cleanup
<code>loans</code> (Management)	<code>management.js:173</code>	Entire collection (Unbounded)	No Cleanup
<code>customers</code> (Management)	<code>management.js:193</code>	Entire collection (Unbounded)	No Cleanup

B. One-Time Fetch Operations (`.get()` & REST Endpoints)

METHOD NAME	TARGET RESOURCE	IMPLEMENTATION IN <code>FIREBASE-CONFIG.JS</code>	CALLING INTERVALS
<code>getLoans()</code>	<code>loans</code>	REST GET <code>/documents/loans</code> + Fallback <code>db.collection('loans').get()</code>	Every 5s in <code>app_gold.js</code> + Every 5s in <code>management.js</code>
<code>getDeletedLoanIds()</code>	<code>deleted_loans</code>	REST GET <code>/documents/deleted_loans</code> + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code>
<code>getDailyRates()</code>	<code>rates/today</code>	REST GET <code>rates/today</code> + REST GET <code>settings/dailyRates</code> + SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code>
<code>getSettings()</code>	<code>settings/branchSeeds</code>	REST GET + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code>
<code>getRules()</code>	<code>settings/rulesMaster</code>	REST GET + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code>
<code>getBranchesList()</code>	<code>settings/branchesList</code>	REST GET + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code> + Every 5s in <code>management.js</code>
<code>getValuersList()</code>	<code>settings/valuersList</code>	REST GET + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code> + Every 5s in <code>management.js</code>
<code>getProductsList()</code>	<code>settings/productsList</code>	REST GET + Fallback SDK <code>.get()</code>	Every 5s in <code>app_gold.js</code> + Every 5s in <code>management.js</code>
<code>getAuditLogs()</code>	<code>audit_logs</code>	SDK <code>.orderBy('timestampMs', 'desc').limit(300).get()</code> + REST	Every 5s in <code>management.js</code>
<code>updateDeviceHeartbeat()</code>	<code>active_sessions/{sid}</code>	SDK <code>doc(sessionId).get()</code> prior to set	Every 30s in <code>app_gold.js</code>

3. Security & Multi-Tenancy Flaws in Current Architecture

Critical Security Finding (`firestore.rules`):

Line 5-7 in `firestore.rules`: `match /{document=**} { allow read, write: if true; }`

The production database currently has **completely open read and write access**. Any employee or connected client can read, update, or delete loans, rates, and audit logs belonging to any other branch.

- **No Branch-Level Isolation:** Branch managers from Branch 01 (Azad Chowk) can read and modify loan applications created by Branch 17 (Rajkot).
- **Client-Side Master Overwrite Risk:** Every 5 seconds, if a client's local storage contains master records, `syncCloudData()` attempts to push them back up via `saveSettings()`, `saveRules()`, and `saveValuersList()`, leading to race conditions between branch terminals.
- **No Immutability on Security Audit Logs:** Audit log documents in `audit_logs` can be deleted or overwritten by anyone with client access.

4. Supabase & PostgreSQL Migration Roadmap

Migrating to PostgreSQL + Supabase will resolve the root architectural problems at the database engine level, not just swap API calls.

AREA	CURRENT FIRESTORE IMPLEMENTATION	TARGET SUPABASE (POSTGRESQL) DESIGN
Data Synchronization	Aggressive 5-second polling (<code>setInterval</code>) across all 120+ client terminals.	Event-driven Supabase Realtime WebSocket channels subscribed only to relevant events (rates change, central announcements). Zero polling loops.
Branch Data Isolation	Full collection downloaded to client; filtered via JavaScript in browser memory.	Postgres Row Level Security (RLS): <code>WHERE branch_id = auth.jwt() ->> 'branch_id'</code> enforced automatically by PostgreSQL.
Query Pagination	Unbounded full table reads on <code>loans</code> and <code>customers</code> .	Explicit server-side SQL pagination: <code>.select('*').eq('branch_id', id).order('created_at', { ascending: false }).range(0, 49) .</code>
Device Presence / Heartbeat	30s read-before-write to <code>active_sessions</code> document generating 345k reads/day.	Supabase Realtime Presence (in-memory WebSocket presence tracking, 0 database disk reads) with optional SQL <code>UPSERT</code> without read check.
Listener Lifecycle	Ignored unsubscribe handlers leaking memory and connections.	Managed channel tracking with explicit <code>supabase.removeChannel(channel)</code> on component unmount and view switches.
Audit Log Security	Open collection modifiable by any authenticated or anonymous client.	Postgres <code>APPEND-ONLY</code> table with RLS policy <code>FOR INSERT WITH CHECK (true)</code> and <code>FOR UPDATE, DELETE USING (false)</code> .

5. Next Steps for Phase 2

With Phase 1 complete, the upcoming **Phase 2 (Schema Design)** will produce:

1. Normalized relational table schemas (`branches` , `users` , `customers` , `loans` , `loan_ornaments` , `rates` , `valuers` , `products` , `rules_master` , `audit_logs`).
2. Foreign keys, constraints, and composite B-tree indexes for fast queries.
3. Complete DDL SQL definitions presented for review prior to execution.